

Brief Summary of Week 6

Objective

Understand Spark architecture and perform efficient data processing using transformations, filtering, schema handling, and optimized file formats.

What I Learn?

- What is Spark Architecture and how does it work?
- What is Lazy Evaluation and DAG?
- How to read CSV and Parquet files in Spark?
- How to perform filtering and selection on dataframes?
- How to modify dataframes by renaming columns, casting data types, and adding new columns?
- How to handle null values efficiently?
- How to perform wide transformations and understand shuffle?
- How to build a complete Spark data pipeline?
- How to save processed data into CSV and Parquet formats?
- Why show () is preferred over collect() for large datasets?

Step 1 – Spark Architecture

What is Spark Architecture?

Apache Spark follows a distributed architecture where the **Driver** creates the Spark application, the **Cluster Manager** allocates resources, and the **Executors** process data in parallel.

Driver:

The Driver is the main program that creates the Spark application, schedules tasks, and coordinates the overall execution.

Cluster Manager:

The Cluster Manager allocates resources (CPU and memory) to the Spark application and manages the available cluster resources.

Executors:

Executors are worker processes that execute tasks assigned by the Driver and store the processed data in memory or disk.

Execution Modes-

Local Mode:

Spark runs on a single machine and is mainly used for learning, testing, and development.

Cluster Mode:

Spark runs on multiple machines (cluster) where the Driver and Executors work together to process large datasets in parallel.

Standard Code for Session Creation -

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("Assignment").getOrCreate()
```

Conclusion- In this step 1, I had created a SparkSession to initialize the Spark application and establish the entry point for performing all DataFrames operations in PySpark.

Step 2 - Lazy Evaluation and DAG (Lineage Graph)

What is Lazy Evaluation?

It means Spark does not execute transformations immediately. It waits until an action (such as show(), count(), or collect()) is called. During this time, Spark creates a DAG (Directed Acyclic Graph), also known as a Lineage Graph, to optimize the execution plan and improve performance.

Work Performed:

- Learned how Spark delays execution until an action is performed.
- Understood how Spark uses a DAG (Lineage Graph) to optimize task execution and reduce unnecessary computations.

Example:

When filter() or select() is used, Spark does not process the data immediately. The operations are executed only when an action like show() or count() is called, making the execution more efficient.

Step 3- Read data from files (CSV, Parquet) with proper schema handling.

What is Reading Data?

Spark can read data from different file formats such as CSV and Parquet. Proper schema handling ensures that each column is assigned the correct data type, making data processing more accurate and efficient.

Work Performed-

- Created a SparkSession to initialize the Spark application.
- Loaded datasets from CSV and Parquet files.
- Used header=True and inferSchema=True to automatically detect column names and data types.
- Verified the schema using printSchema() and displayed sample records using show().

Conclusion-

- CSV and Parquet files were loaded successfully.
- Spark automatically identified the schema.
- The dataset structure and sample records were displayed for further processing.

Step 4 – Filtering and Selection

In this step, the dataset was filtered to retain only the records that satisfied the specified condition. After filtering, only the required columns were selected from the DataFrame. This process reduces the amount of unnecessary data and improves processing.

Conclusion-

The DataFrame was successfully filtered based on the given conditions such as (where Category is equal to Electronics), and (where status is Completed and amount>1000) only the required columns (id, price and category) were selected for further processing and analysis.

Step 4 – Applying Transformations

In this, I modified DataFrames by Renaming Columns, Casting Data Types, and Adding New Columns

- The column product_price was renamed to price.
- The price column's data type was converted from String to Double using the cast() function, to remove further errors.
- A new column named final_price was created using the withColumn() method. The new column was calculated by multiplying the base_price by 1.18, representing the final price.\

Step 5 – Handling Null Values

In this task, the dataset was cleaned by handling null values and filtering only the required records. Null values were removed using the dropna() function to ensure data quality and prevent errors during analysis.

```
# Remove rows containing null values  
clean_df = df.dropna()
```

Step 6 - Performing Wide Transformations and Performance Concepts (Shuffle and Predicate Pushdown)

In this task, the concepts of wide transformations, shuffle, and predicate pushdown were studied. Wide transformations, such as groupBy(), join(), and distinct(), require data to be redistributed across partitions. This redistribution process is called shuffle, where Spark transfers data between executors to ensure that records with the same key are processed together.

Another important optimization technique is Predicate Pushdown. In predicate pushdown, Spark pushes filter conditions down to the data source (such as Parquet) so that only the required rows are read into memory.

Conclusion-

The groupBy() operation demonstrated a wide transformation, where Spark performed a shuffle to redistribute data before aggregation. The filter() operation illustrated the concept of predicate pushdown, allowing Spark to read only the required records from supported file formats such as Parquet

Step 7 - Building a Complete Spark Data Pipeline

In this task, a complete Spark data pipeline was developed to process data efficiently. The pipeline followed the standard Extract, Transform, and Load (ETL) workflow by reading data from a source file, applying multiple transformations, filtering the required records, and writing the processed data to an output location.

Step 8- Saving Processed Data into CSV and Parquet Formats

Step 9 - Why show() is Preferred Over collect() for Large Datasets?

In Apache Spark, both `show()` and `collect()` are actions, but they serve different purposes. The `show()` method is primarily used to display a small number of rows from a `DataFrame` for inspection, whereas `collect()` retrieves all the data from the distributed cluster and brings it to the driver program.

For large datasets, `show()` is preferred because it displays only a limited number of records (20 rows by default). In contrast, `collect()` loads the entire dataset into the driver's memory. If the dataset is very large, it can consume excessive memory, degrade performance, or even cause an `OutOfMemoryError`.